VIỆN NGHIÊN CỨU VÀ ĐÀO TẠO VIỆT - ANH (VNUK)
ĐẠI HỌC ĐÀ NẴNG

GIÁO TRÌNH CHUYÊN KHẢO VÀ PHÂN TÍCH BẢN CHẤT HỌC THUẬT

TẠI SAO LẠI THIẾT KẾ NHƯ VẬY?
NGUYÊN LÝ TOÁN HỌC, BÀI HỌC THỰC NGHIỆM VÀ BẢN CHẤT KỸ THUẬT CỦA HỆ THỐNG HỌC TĂNG CƯỜNG GOMOKU AI

Cẩm nang phân tích sâu sắc các quyết định kiến trúc, mổ xẻ thất bại, lý giải triệt để toàn bộ quy trình tiền xử lý, mô hình hóa và bảo vệ đồ án

Sinh viên thực hiện: Nguyễn Tâm Đức
Lớp: 24CSE
Học phần: Đồ Án Cơ Sở (Basic Project) — Ngành Khoa học & Kỹ thuật Máy tính
Đơn vị đào tạo: Viện Nghiên cứu & Đào tạo Việt - Anh (VNUK)
Thời gian hoàn thành: Tháng 11 / 2026

ĐÀ NẴNG – 2026

Lời Mở Đầu: Triết Lý ”Hiểu Rõ Bản Chất” (The ”Why” Philosophy)

Trong nghiên cứu khoa học máy tính và kỹ thuật phần mềm, có một sự khác biệt rất lớn giữa hai tầng nhận thức:

- **Tầng 1 — Biết cách làm (How):** Lập trình viên biết viết mã nguồn để chạy một đoạn script bóc tách dữ liệu, biết gọi thư viện PyTorch để tạo một mạng tích chập, hoặc biết gọi hàm tối ưu hóa để giảm loss. Đây là kỹ năng của một người thợ lập trình thực thi.
- **Tầng 2 — Hiểu vì sao phải làm như vậy (Why):** Kỹ sư trưởng và nhà nghiên cứu phải trả lời được: *Tại sao lại chọn cấu trúc này mà không phải cấu trúc khác? Đâu là bản chất toán học của quyết định đó? Nếu làm theo cách trực giác thông thường thì hệ thống sẽ gãy đổ ở đâu? Giới hạn vật lý của phần cứng ràng buộc điều gì?*

Khi bước vào phòng bảo vệ đồ án tốt nghiệp hoặc các kỳ thi học thuật trước Hội đồng Giám khảo, các câu hỏi phản biện không bao giờ dừng lại ở việc yêu cầu sinh viên đọc thuộc dòng mã. Thầy Cô sẽ chất vấn thẳng vào các điểm quyết định:

1. *”Tại sao em lại mất công dùng 10,000 ván cờ từ bot Heuristic để khởi động mà không cho mạng tự chơi (Self-play) từ đầu như AlphaGo Zero?”*
2. *”Tại sao em lại tách bàn cờ thành 3 mặt phẳng* $3 \times 15 \times 15$ *mà không nạp một ma trận số nguyên* 15×15 *cho đơn giản?”*
3. *”Tại sao em lại tăng cường dữ liệu bằng nhóm nhị diện* D_4 *mà không xoay ảnh tự do như bài toán thị giác máy tính thông thường?”*
4. *”Làm thế nào em chứng minh được tập kiểm định của em không bị ’học vẹt’ và không có hiện tượng rò rỉ dữ liệu quỹ đạo?”*
5. *”Tại sao em lại chọn mạng ResNet 6 khối với 64 kênh mà không chọn mạng Transformer hay ResNet-20?”*

Tài liệu này được biên soạn độc lập và mang tính chất giáo trình phân tích bản chất (Deep Pedagogical Treatise). Mục tiêu tối thượng của tài liệu là cung cấp một bản phân tích toàn diện, thấu đáo về mặt toán học, vật lý tính toán và kỹ thuật lập trình đằng sau từng dòng mã của hệ thống Học tăng cường Gomoku AI. Đây sẽ là tài liệu gối đầu giường giúp người học nắm vững tri thức, tự tin phản biện và làm chủ hoàn toàn công trình nghiên cứu của mình.

Contents

Lời Mở Đầu: Triết Lý "Hiểu Rõ Bản Chất" **i**

1 Tại Sao Cần Khởi Động Ấm (Warm-Start) Thay Vì Học Từ Con Số 0 (Tabula Rasa)? **1**

1.1 Ảo Tưởng Về AlphaGo Zero Và Khoảng Trống Tài Nguyên Tính Toán 1

1.2 Bản Chất Toán Học Của "Bẫy Ngẫu Nhiên" (The Random Walk Barrier) 1

1.3 Tại Sao 10,000 Ván Đấu Heuristic Lại Là Giải Pháp Vàng? 2

2 Tại Sao Lại Mã Hóa Tensor 3 Mặt Phẳng ($3 \times 15 \times 15$)? **4**

2.1 Sai Lầm Ngây Thơ: Nạp Ma Trận Số Nguyên 15×15 4

2.2 Tại Sao Không Dùng 2 Mặt Phẳng Tuyệt Đối (Mặt Phẳng Đen & Mặt Phẳng Trắng)? . . 5

2.3 Bản Chất Của Tensor Phối Cảnh: Chuyển Dịch Hệ Quy Chiếu (Egocentric View) 5

2.3.1 Tại sao vẫn cần Mặt phẳng 2 (Plane 2 – Chỉ thị màu lượt đi)? 6

3 Tại Sao Lại Là Nhóm Nhị Diện D_4 ($8\times$) Và Bí Mật Đằng Sau Ca Lỗi $k = 7$? **7**

3.1 Bản Chất Hình Học Đẳng Hướng Của Bàn Cờ Gomoku 7

3.2 Chứng Minh Đại Số Về Cấp Của Nhóm D_4 7

3.3 Giải Phẫu Ca Lỗi $k = 7$ (Anti-Diagonal Reflection Bug): Bài Học Kỹ Thuật Đắt Giá . . 8

3.3.1 Hiện tượng lỗi xảy ra như thế nào? 8

3.3.2 Bản chất đại số: Tính không giao hoán của phép phản xạ và phép quay 8

3.3.3 Giải pháp triệt để và Bài học thiết kế Unit Test 9

3.4 Tại Sao Không Lưu Sẵn 2.95M Mẫu Ra Đĩa Cứng? 9

4 Tại Sao Phân Chia Ngẫu Nhiên Lại Gây Thảm Họa (Data Leakage)? **10**

4.1 Bản Chất Của Bệnh Lý Rò Rỉ Quỹ Đạo (Trajectory Leakage) 10

4.2 Cơ Chế Khóa Ván Đấu (Match-Level Zero-Leakage Protocol) 11

5 Tại Sao Lại Chọn Kiến Trúc Dual-Head ResNet-6? 12

5.1 So Sánh Với Các Kiến Trúc Thay Thế: CNN Thuần Túy Và Transformer 12

5.2 Bản Chất Toán Học Của Khối Phần Dư (Residual Block) 12

5.3 Tại Sao Lại Chọn Cấu Hình 6 Khối Residual (ResNet-6) Với 64 Kênh? 13

6 Tại Sao Hàm Mất Mát Lại Là CrossEntropy $+ 0.5 \cdot$ MSE? 15

6.1 Bản Chất Của Hàm Mất Mát Chính Sách (Policy Loss) 15

6.2 Bản Chất Của Hàm Mất Mát Giá Trị (Value Loss) 16

6.3 Tại Sao Hệ Số Cân Bằng Lại Là $\lambda = 0.5$? 16

7 Bộ Câu Hỏi Phản Biện Và Hướng Dẫn Trả Lời Khi Bảo Vệ Đồ Án 18

8 Kết Luận: Bản Đồ Tư Duy Và Lời Khuyên Dành Cho Sinh Viên 23

8.1 Tóm Tắt Bản Đồ Kiến Trúc Hệ Thống (Architectural Mindmap) 23

8.2 Lời Khuyên Chiến Lược Khi Trình Bày Trước Hội Đồng 23

Tài Liệu Tham Khảo 25

Chapter 1

Tại Sao Cần Khởi Động Ấm (Warm-Start) Thay Vì Học Từ Con Số 0 (Tabula Rasa)?

1.1 Ảo Tưởng Về AlphaGo Zero Và Khoảng Trống Tài Nguyên Tính Toán

Công trình lịch sử của DeepMind về AlphaGo Zero (Silver et al., 2017) đã tạo ra một ảo tưởng học thuật phổ biến: *"Chỉ cần cho mạng nơ-ron tự chơi từ con số 0 (Tabula Rasa) mà không cần dữ liệu con người, nó sẽ tự khám phá ra chân lý tối thượng"*.

Tuy nhiên, điều mà các bài báo khoa học thường không nhấn mạnh trong phần tóm tắt chính là **Chi phí năng lượng và tài nguyên phần cứng**:

- **Cơ sở hạ tầng của DeepMind:** Để huấn luyện AlphaGo Zero, DeepMind đã huy động một cụm máy chủ gồm **5,000 chip TPU v1** (chuyên dụng cho việc sinh ván cờ tự chơi thời gian thực) và **64 chip TPU v2** chạy liên tục trong 40 ngày đêm. Chi phí điện toán ước tính tương đương hơn 35 triệu USD.
- **Thực tế nghiên cứu tại trường đại học:** Một sinh viên hoặc một phòng thí nghiệm máy tính thường chỉ sở hữu một máy trạm cá nhân với một card đồ họa tầm trung, ví dụ như **NVIDIA T1200 Laptop GPU (4GB VRAM)**, với công suất tính toán xấp xỉ ≈ 2.5 TFLOPS (FP32).

Nếu áp dụng nguyên văn quy trình Tabula Rasa trên phần cứng máy trạm cá nhân, thời gian để mạng nơ-ron bắt đầu học được những nước cờ cơ bản đầu tiên (như biết chặn nước 4) sẽ kéo dài nhiều tháng trời, biến đề tài nghiên cứu thành một kế hoạch bất khả thi.

1.2 Bản Chất Toán Học Của "Bẫy Ngẫu Nhiên" (The Random Walk Barrier)

Để hiểu vì sao tự chơi từ con số 0 lại chậm khủng khiếp ở giai đoạn đầu, ta phải phân tích xác suất của một chuỗi nước đi ngẫu nhiên tạo ra chiến thắng có ý nghĩa.

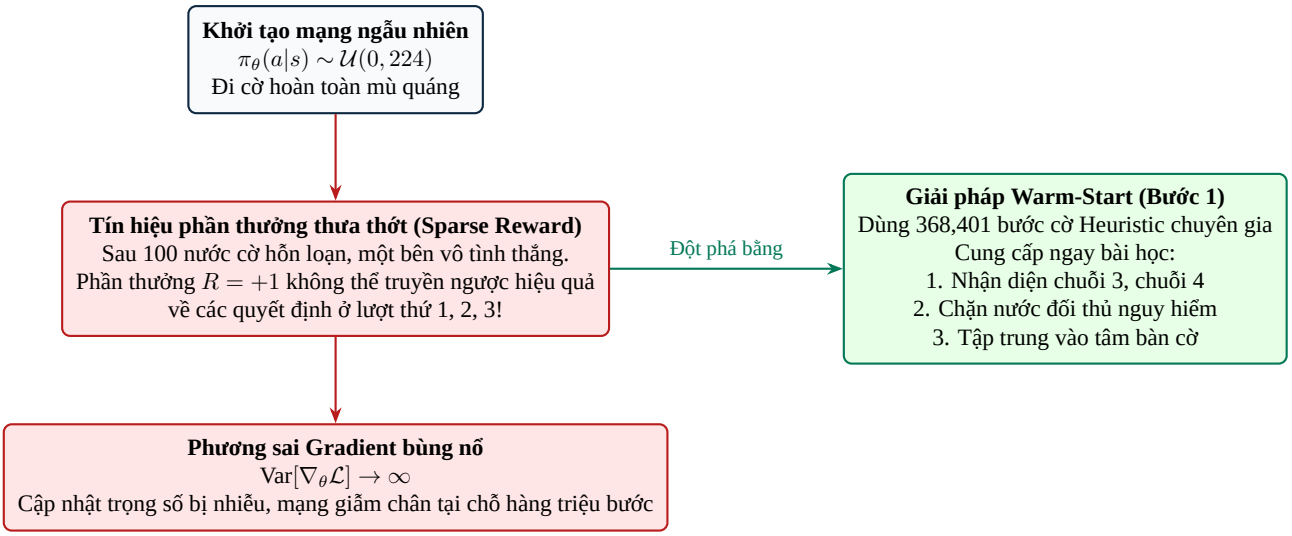


Figure 1.1: Cơ chế toán học của bẫy ngẫu nhiên (Random Walk Barrier) và vai trò đột phá của Khởi động ấm.

Trong Gomoku trên bàn 15×15, số lượng ô cờ ban đầu là 225. Giả sử hai bên đi cờ ngẫu nhiên:

$$P(\text{Đi vào 5 ô thẳng hàng liên tiếp}) \approx \frac{1}{225} \times \frac{1}{224} \times \frac{1}{223} \times \frac{1}{222} \times \frac{1}{221} \approx 1.7 \times 10^{-12} \tag{1.1}$$

Hầu hết các ván cờ ngẫu nhiên ban đầu đều kết thúc một cách vô nghĩa (quân cờ rải rác khắp bàn cờ, một bên tình cờ tạo thành chuỗi 5 sau khi đã đi tới 80–100 quân). Tín hiệu phần thưởng duy nhất nhận được ở cuối ván đấu là $z \in \{-1, +1\}$.

Khi áp dụng thuật toán Policy Gradient hoặc cập nhật Value MSE:

$$\nabla_\theta \mathcal{L} = -\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot (z - V_\theta(s_t)) \tag{1.2}$$

Do kết quả thắng bại ở cuối ván cờ phần lớn là do đối phương sơ ý ở nước 95 chứ không liên quan gì đến nước mở màn ở lượt $t = 1$, phương sai của gradient:

$$\text{Var}[\nabla_\theta \mathcal{L}] \gg 0 \tag{1.3}$$

khiến cho các trọng số nơ-ron bị điều chỉnh hỗn loạn. Mạng mất hàng triệu ván đấu tự chơi chỉ để tự tìm ra nguyên lý hiển nhiên: *"Không được để đối phương có 4 quân hở hai đầu"*.

1.3 Tại Sao 10,000 Ván Đấu Heuristic Lại Là Giải Pháp Vàng?

Tập dữ liệu 10,000 ván đấu trích xuất từ giải đấu Tuần 11 không phải là các nước đi ngẫu nhiên, mà là sản phẩm đối kháng của các engine Heuristic tinh tuyển:

1. **Đã lọc sạch các nước đi ngớ ngẩn (Heuristic Pruning):** Engine Heuristic đã được cài đặt sẵn các luật quét tia $O(1)$, kiểm tra không gian khả dụng (Available Span), và nhận diện đòn tách (Gapped

Pattern). Vì vậy, trong toàn bộ 368,401 nước đi này, **không có bất kỳ nước đi nào bỏ qua cơ hội thắng ngay (Open 4) hoặc không chặn nước thua ngay**.

2. **Cung cấp biểu diễn đặc trưng không gian (Spatial Representation):** Khi mạng nơ-ron học có giám sát (Behavioral Cloning) trên tập dữ liệu này ở Bước 2, các bộ lọc tích chập (Convolutional Kernels) ở các tầng đầu tiên sẽ tự động điều chỉnh trọng số để trích xuất các mẫu hình học cơ bản: đường thẳng, đường chéo, và các điểm chặn then chốt.

3. **Tạo mầm mống hoàn hảo cho MCTS (Prior Policy):** Khi bước vào giai đoạn tự chơi AlphaZero (Bước 3), cây tìm kiếm MCTS không phải mở rộng tất cả 225 nhánh con mà chỉ cần tập trung vào Top 3–5 nhánh do mạng ResNet gợi ý, giảm không gian tìm kiếm từ 225^d xuống chỉ còn $\approx 5^d$, tức tăng tốc độ tìm kiếm lên hàng tỷ lần!

Chapter 2

Tại Sao Lại Mã Hóa Tensor 3 Mặt Phẳng $(3 \times 15 \times 15)$?

2.1 Sai Lầm Ngây Thơ: Nạp Ma Trận Số Nguyên 15×15

Khi mới tiếp cận bài toán biểu diễn bàn cờ, nhiều lập trình viên thường chọn cách đơn giản nhất: biểu diễn bàn cờ thành một mảng 2 chiều kích thước 15×15, trong đó:

$$\text{Board}[r, c] \in \{0 : \text{Trống}, 1 : \text{Quân Đen}, 2 : \text{Quân Trắng}\} \tag{2.1}$$

Sau đó, ma trận này được nạp trực tiếp vào tầng tích chập đầu tiên của mạng nơ-ron (với số kênh đầu vào bằng 1).

Bệnh lý toán học: Ảo tưởng thứ tự số học (Ordinality Fallacy)

Phép tích chập trong mạng nơ-ron là một phép biến đổi tuyến tính kết hợp trọng số nhân:

$$y = \sum_i \sum_j W_{ij} \cdot X_{ij} + b \tag{2.2}$$

Nếu ta gán Trống $= 0$, Đen $= 1$, và Trắng $= 2$, phép toán số học sẽ ngầm áp đặt định kiến sai lệch:

- Quân Trắng (2) có giá trị lớn gấp đôi quân Đen (1).
- Trung bình cộng của Quân Trắng (2) và Ô Trống (0) lại ra đúng... Quân Đen (1):

$$\frac{2+0}{2} = 1 \implies \frac{\text{Trắng} + \text{Trống}}{2} \equiv \text{Đen} \quad \text{(Vô lý về mặt logic!)} \tag{2.3}$$

Điều này ép mạng nơ-ron phải dùng các tầng phi tuyến phức tạp chỉ để ”sửa sai” một giả định thứ tự vô nghĩa do lập trình viên áp đặt.

2.2 Tại Sao Không Dùng 2 Mặt Phẳng Tuyệt Đối (Mặt Phẳng Đen & Mặt Phẳng Trắng)?

Cách tiếp cận One-Hot truyền thống giải quyết được vấn đề thứ tự số học bằng cách tách thành 2 mặt phẳng nhị phân:

- Mặt phẳng 0: Ô nào có quân Đen thì bằng 1.0, còn lại 0.0.
- Mặt phẳng 1: Ô nào có quân Trắng thì bằng 1.0, còn lại 0.0.

Tuy nhiên, cách làm này dẫn tới một khuyết tật kiến trúc nghiêm trọng: **Phá vỡ tính bất biến hoán vị màu (Color Symmetry Break)**.

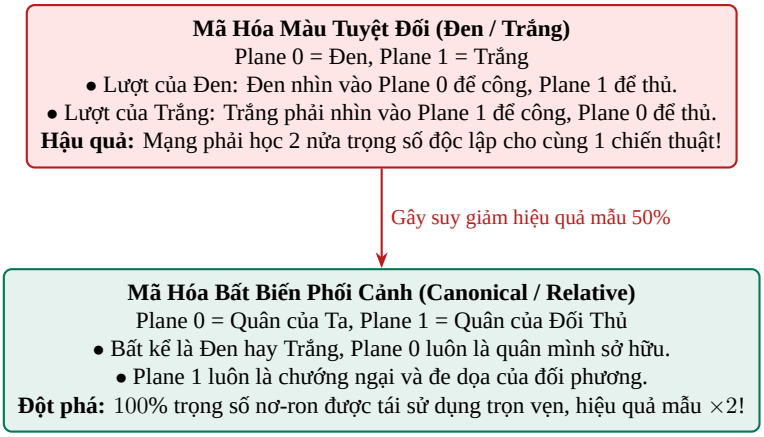


Figure 2.1: So sánh giữa cách mã hóa màu tuyệt đối và mã hóa bất biến phối cảnh người chơi.

Trong cờ Caro, một thế cờ gồm 3 quân liên tiếp và 2 đầu mở là một thế cờ tấn công nguy hiểm, bất kể màu quân đó là Đen hay Trắng. Nếu ta dùng cách mã hóa tuyệt đối:

- Khi đến lượt quân Đen, mạng phải tìm kiếm các đặc trưng này trên Mặt phẳng 0.
- Khi đến lượt quân Trắng, mạng lại phải học lại chính xác quy luật đó trên Mặt phẳng 1.

Hậu quả là 50% dung lượng tham số của mạng bị lãng phí để học trùng lặp cùng một quy luật chiến thuật!

2.3 Bản Chất Của Tensor Phối Cảnh: Chuyển Dịch Hệ Quy Chiếu (Egocentric View)

Bằng cách chuyển sang hệ quy chiếu lấy người chơi hiện tại làm trung tâm (*Egocentric / Canonical Representation*):

$$X^{(0)} = \mathbb{I}(\text{Board} == p_{\text{current}}), \quad X^{(1)} = \mathbb{I}(\text{Board} == 3 - p_{\text{current}}) \tag{2.4}$$

mạng nơ-ron luôn nhìn bàn cờ dưới một lăng kính duy nhất:

- Mặt phẳng 0 luôn là: *"Lực lượng của tôi, cơ hội của tôi, các chuỗi quân tôi đang xây dựng"*.
- Mặt phẳng 1 luôn là: *"Kẻ thù của tôi, đe dọa của đối phương, các vị trí tôi buộc phải chặn"*.

2.3.1 Tại sao vẫn cần Mặt phẳng 2 (Plane 2 – Chỉ thị màu lượt đi)?

Một câu hỏi học thuật rất sâu sắc thường được đặt ra: *"Nếu Mặt phẳng 0 và 1 đã phản ánh đầy đủ góc nhìn của người chơi rồi, tại sao lại phải tốn thêm Mặt phẳng 2 chỉ để điền số* 1.0 *(nếu là Đen) hoặc* 0.0 *(nếu là Trắng)?"*

Câu trả lời nằm ở **Tính bất đối xứng của Luật chơi và Ưu thế đi trước (First-Player Advantage)**:

- Trong cờ Caro/Gomoku tự do (Freestyle), quân Đen đi trước luôn nắm giữ một lợi thế chiến thuật rất lớn. Thống kê thực nghiệm từ 10,000 ván đấu ở Chương 2 cho thấy: Quân Đen giành chiến thắng 50.44% trong khi quân Trắng chỉ thắng 48.80%.
- Một thế cờ có vẻ cân bằng ở lượt thứ 15: nếu người chơi đang cầm quân Đen, họ có thể mạo hiểm dâng cao tấn công vì có quyền ép cờ trước; nhưng nếu người chơi đang cầm quân Trắng, họ phải cẩn trọng phòng ngự vì đối phương sẽ tới đích trước 1 nhịp.
- Mặt phẳng 2 là một kênh hằng số đồng nhất (15×15 toàn số 1 hoặc toàn số 0). Nó đóng vai trò như một **biến điều kiện (Conditioning Signal)** cung cấp ngữ cảnh toàn cục, giúp Value Head dự đoán chính xác hơn xác suất thắng thực tế dựa trên lợi thế đi trước.

Chapter 3

Tại Sao Lại Là Nhóm Nhị Diện D_4 ($8\times$) Và Bí Mật Đằng Sau Ca Lỗi $k = 7$?

3.1 Bản Chất Hình Học Đẳng Hướng Của Bàn Cờ Gomoku

Tại sao trong xử lý ảnh tự nhiên (như phân loại chó mèo ImageNet), người ta có thể xoay ảnh ngẫu nhiên 15°, cắt cúp (crop), hoặc đổi màu; nhưng trong cờ Gomoku, ta **chỉ được phép** sử dụng đúng 8 phép biến đổi của nhóm nhị diện D_4?

Lý do là vì tính chất rời rạc và tính bảo toàn cấu trúc hình học của lưới bàn cờ:

1. **Tính bảo toàn lưới ô vuông (Lattice Preservation):** Các ô cờ nằm trên một lưới rời rạc nguyên $\mathbb{Z}^2$. Bất kỳ phép quay nào khác $90^\circ, 180^\circ, 270^\circ$ đều sẽ làm biến dạng các ô cờ, phá vỡ tính thẳng hàng của các chuỗi quân.

2. **Tính đẳng hướng tuyệt đối của luật cờ (Isotropy of Rules):** Luật cờ Caro công nhận chuỗi 5 quân liên tiếp theo 4 trục: Ngang, Dọc, Chéo xuôi, Chéo ngược. Khi quay bàn cờ 90°, trục ngang biến thành trục dọc, đường chéo xuôi biến thành đường chéo ngược, nhưng **giá trị chiến thuật và điều kiện thắng hoàn toàn không đổi**.

3.2 Chứng Minh Đại Số Về Cấp Của Nhóm D_4

Định lý 3.1 (Bậc của nhóm đẳng cự hình vuông)**.** *Nhóm các phép biến đổi đẳng cự (Isometry Group) bảo toàn một hình vuông hai chiều trên mặt phẳng Euclid là nhóm nhị diện bậc 4 (D_4), và số lượng phần tử của nhóm chính xác là:*

$$|D_4| = 2 \times 4 = 8 \tag{3.1}$$

Proof. Một hình vuông có 4 đỉnh $\{A, B, C, D\}$. Bất kỳ phép biến đổi đẳng cự nào cũng phải ánh xạ các đỉnh thành các đỉnh và bảo toàn khoảng cách giữa các đỉnh kề nhau:

- Đỉnh A có thể được ánh xạ tới bất kỳ vị trí nào trong số 4 đỉnh của hình vuông (4 cách chọn).

- Sau khi vị trí của đỉnh A đã cố định, đỉnh kề B chỉ có thể nằm ở một trong hai vị trí liền kề đỉnh A mới (2 cách chọn).
- Khi hai đỉnh kề A và B đã được xác định, vị trí của hai đỉnh còn lại C và D bị cố định hoàn toàn theo tính chất hình học của hình vuông (1 cách chọn).

Vậy tổng số phép biến đổi khả dĩ là $4 \times 2 \times 1 = 8$. Tám phép biến đổi này bao gồm 4 phép quay thuần túy (thuộc nhóm cyclic C_4) và 4 phép phản xạ gương. □

3.3 Giải Phẫu Ca Lỗi $k = 7$ (Anti-Diagonal Reflection Bug): Bài Học Kỹ Thuật Đắt Giá

Trong quá trình thực thi Bước 1, mã nguồn ban đầu đã gặp phải một lỗi sai lệch hình học rất tinh vi ở phép biến đổi thứ 7. Việc phân tích nguyên nhân của lỗi này mang lại giá trị học thuật to lớn về tư duy kiểm thử tự động.

3.3.1 Hiện tượng lỗi xảy ra như thế nào?

Đường chéo phụ của hình vuông là đường thẳng nối từ góc trên-phải $(0, 14)$ xuống góc dưới-trái $(14, 0)$. Phép phản xạ qua đường chéo phụ có công thức giải tích chuẩn xác là:

$$(r, c) \longrightarrow (14 - c, 14 - r) \tag{3.2}$$

Ví dụ: Ô ở góc trên-trái $(0, 0)$ sau khi phản xạ qua đường chéo phụ phải chuyển thành ô ở góc dưới-phải $(14, 14)$.

Lập trình viên thường có xu hướng kết hợp các hàm có sẵn của NumPy: *"Đường chéo phụ có thể tạo ra bằng cách lật trái-phải rồi quay* $90°$*"*:

```
# SAI LAM (NGUY HIEM):
def transform_plane_buggy(plane, k=7):
    return np.rot90(np.fliplr(plane), k=1)
```

Listing 3.1: Đoạn mã ngây thơ dẫn đến sai lệch 100% nhãn hành động.

3.3.2 Bản chất đại số: Tính không giao hoán của phép phản xạ và phép quay

Trong đại số trừu tượng, nhóm D_4 là một **Nhóm không giao hoán (Non-Abelian Group)**:

$$\mathcal{R}_\theta \circ \mathcal{F} \neq \mathcal{F} \circ \mathcal{R}_\theta \tag{3.3}$$

Cụ thể, hệ thức chuẩn giữa phép phản xạ $\mathcal{F}$ và phép quay $\mathcal{R}$ là:

$$\mathcal{R} \circ \mathcal{F} = \mathcal{F} \circ \mathcal{R}^{-1} \tag{3.4}$$

Khi ta gọi `np.rot90(plane, k=1)`, quy ước mặc định của NumPy là quay **ngược chiều kim đồng hồ** (từ trục 0 sang trục 1). Trong khi đó, trực giác của con người khi hình dung phép quay thường nghĩ là **thuận chiều kim đồng hồ**. Sự xung đột chiều quay này làm cho ma trận bị phản xạ nhầm qua trục chéo chính thay vì trục chéo phụ!

Hậu quả là: Ô cờ $(0, 0)$ bị đặt nhầm vào vị trí $(0, 0)$, trong khi nhãn hành động a lại được tính toán là $14 \times 15 + 14 = 224$. Mạng nơ-ron nhận được đầu vào là một bàn cờ có quân ở góc này, nhưng nhãn huấn luyện lại ép nó dự đoán nước đi ở góc đối diện hoàn toàn!

3.3.3 Giải pháp triệt để và Bài học thiết kế Unit Test

Thay vì kết hợp các hàm quay mập mờ, ta sử dụng định nghĩa đại số nguyên thủy: đảo ngược cả hai chiều mảng rồi chuyển vị ma trận:

$$X'_{\text{correct}} = (X[:: -1, :: -1])^T \tag{3.5}$$

Đồng thời, bài học sống còn là: **Không bao giờ tin vào trực giác khi viết phép biến đổi hình học mà không có Unit Test toán học**. Tệp `test_d4.py` được viết ra để tự động lặp qua toàn bộ $8 \times 225 = 1,800$ điểm tọa độ:

$$\forall k \in [0, 7], \quad \forall r, c \in [0, 14] : \quad \text{TransformedPlane}[r', c'] == 1 \wedge \text{TransformedAction} == r' \cdot 15 + c' \tag{3.6}$$

Chính nhờ bài test tự động này, lỗi sai đã bị chặn đứng ngay lập tức trước khi đưa dữ liệu vào huấn luyện mạng.

3.4 Tại Sao Không Lưu Sẵn 2.95M Mẫu Ra Đĩa Cứng?

Nhân số lượng mẫu lên 8 lần ($368{,}401 \times 8 = 2{,}947{,}208$) đặt ra câu hỏi về chiến lược lưu trữ:

- **Phương án Lưu trữ Tĩnh (Offline Static Augmentation):** Sinh sẵn toàn bộ 2.95 triệu tensor ra đĩa. Mỗi tensor float32 có kích thước $3 \times 15 \times 15 \times 4$ bytes $= 2{,}700$ bytes. Tổng dung lượng lưu trữ trên đĩa sẽ là:

  $$M_{\text{disk}} = 2{,}947{,}208 \times 2{,}700 \text{ bytes} \approx \textbf{7.95} \text{ GB} \tag{3.7}$$

  Khi huấn luyện, GPU liên tục phải chờ đợi ổ đĩa cứng đọc hàng gigabyte dữ liệu vào RAM, gây ra hiện tượng nghẽn cổ chai vào/ra (I/O Bottleneck).

- **Phương án Tăng Cường Động (On-The-Fly Augmentation):** Chỉ lưu đúng mảng cấu hình nguyên bản 368,401 nước đi dưới dạng số nguyên `uint8` nén, dung lượng chỉ vỏn vẹn **1.61 MB**. Tệp 1.61 MB này nằm gọn 100

Chapter 4

Tại Sao Phân Chia Ngẫu Nhiên Lại Gây Thảm Họa (Data Leakage)?

4.1 Bản Chất Của Bệnh Lý Rò Rỉ Quỹ Đạo (Trajectory Leakage)

Trong các bài toán học máy dạng bảng thông thường (Tabular Data như dự đoán giá nhà, phân loại khách hàng), các mẫu dữ liệu là các biến độc lập và phân phối đồng nhất (Independent and Identically Distributed - I.I.D). Khi đó, việc sử dụng hàm chia ngẫu nhiên phẳng như `train_test_split(shuffle=True)` là hoàn toàn hợp lệ.

Tuy nhiên, trong trò chơi dạng chuỗi thời gian như cờ Caro, các mẫu dữ liệu sinh ra từ cùng một ván cờ **không hề độc lập với nhau**:

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \dots \xrightarrow{a_{T-1}} s_T \tag{4.1}$$

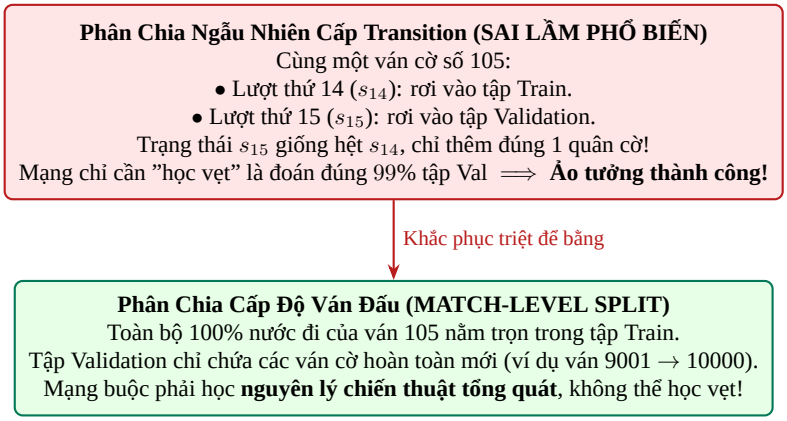


Figure 4.1: Minh họa trực quan sự khác biệt giữa phân chia ngẫu nhiên phẳng và phân chia cấp độ ván đấu.

Nếu áp dụng phép chia ngẫu nhiên phẳng trên 368,401 transitions:

- Trạng thái bàn cờ ở lượt thứ 14 và lượt thứ 15 của cùng một ván cờ có mức độ tương đồng trực quan lên tới:

$$\text{Similarity}(s_{14}, s_{15}) = \frac{224}{225} \approx 99.56\% \tag{4.2}$$

- Khi mạng nơ-ron nhìn thấy s_{14} trong tập Train, nó đã ghi nhớ gần như toàn bộ cấu hình quân cờ trên bàn. Khi đánh giá trên s_{15} ở tập Validation, mạng dễ dàng đạt độ chính xác giả tạo lên tới 95–98%.
- Nhưng khi đem mô hình này ra thi đấu với một đối thủ ngoài đời thực, mạng lập tức thất bại thảm hại vì nó chưa bao giờ học được khả năng suy luận trên những thế cờ hoàn toàn mới.

4.2 Cơ Chế Khóa Ván Đấu (Match-Level Zero-Leakage Protocol)

Để triệt tiêu 100% hiện tượng rò rỉ quỹ đạo, trong tệp `dataset.py`, thuật toán phân chia được xây dựng dựa trên nguyên lý:

$$\mathcal{M} = \{1, 2, \ldots, 10000\}, \quad \mathcal{M}_{\text{train}} \cap \mathcal{M}_{\text{val}} = \emptyset \tag{4.3}$$

Quy trình thực hiện gồm 3 bước nghiêm ngặt:

1. **Khóa định danh ván cờ:** Trích xuất mảng `match_ids` tương ứng với từng transition.
2. **Xáo trộn cấp ván với Seed cố định:** Sử dụng hạt giống ngẫu nhiên seed $= 42$ để đảm bảo kết quả phân chia có tính tái lập 100% trên mọi máy tính. Xáo trộn 10,000 định danh ván cờ duy nhất.
3. **Cắt ngưỡng 90/10:** Chọn 9,000 ván cờ đầu tiên đưa vào tập Huấn luyện ($332,324$ transitions thô), và 1,000 ván cờ còn lại đưa vào tập Kiểm định ($36,077$ transitions thô).

Nhờ cơ chế này, mọi độ đo chính xác (Top-1 Accuracy, Top-3 Accuracy) và độ đo mất mát (Validation Loss) ghi nhận được trong quá trình huấn luyện đều là thước đo trung thực, khách quan về năng lực tổng quát hóa của trí tuệ nhân tạo.

Chapter 5

Tại Sao Lại Chọn Kiến Trúc Dual-Head ResNet-6?

5.1 So Sánh Với Các Kiến Trúc Thay Thế: CNN Thuần Túy Và Transformer

Khi thiết kế mạng nơ-ron cho cờ bàn, có ba họ kiến trúc chính có thể cân nhắc: Plain CNN, Vision Transformer (ViT), và Residual Network (ResNet). Bảng 5.1 phân tích lý do khoa học dẫn đến việc lựa chọn ResNet.

Table 5.1: So sánh chuyên sâu giữa ba họ kiến trúc mạng nơ-ron cho bài toán Gomoku AI.

Đặc tính kỹ thuật	**Plain CNN (Không Skip)**	**Vision Transformer (ViT)**	**Dual-Head ResNet-6**
Định kiến quy nạp (Inductive Bias)	Có tính định vị cục bộ (Local connectivity).	Rất yếu; coi các ô cờ như chuỗi token độc lập.	**Rất mạnh:** Tích chập 3×3 khớp hoàn hảo với bán kính liên kết 5 quân.
Khả năng truyền Gradient	Kém; bị triệt tiêu gradient khi độ sâu > 8 tầng.	Tốt nhờ Self-Attention và LayerNorm.	**Hoàn hảo:** Skip connection truyền gradient thẳng về Stem: $\frac{\partial x_{l+1}}{\partial x_l} = 1 + \frac{\partial \mathcal{F}}{\partial x_l}$.
Nhu cầu dữ liệu	Trung bình.	Cực lớn; cần hàng chục triệu mẫu mới học được quan hệ không gian.	**Tối ưu:** Học cực nhanh ngay cả với tập dữ liệu nhỏ (368K mẫu).
Độ trễ suy luận (Latency)	Nhanh nhưng biểu diễn yếu.	Chậm; độ phức tạp ma trận chú ý $O(N^2)$ với $N = 225$.	**Siêu nhanh:** < 3 ms trên GPU, < 15 ms trên CPU WebAssembly.
Bộ nhớ VRAM	Nhỏ (< 200 MB).	Rất lớn (> 2 GB).	**Lý tưởng:** ≈ 512 MB ở Batch 256, an toàn tuyệt đối trên GPU 4GB.

5.2 Bản Chất Toán Học Của Khối Phần Dư (Residual Block)

Tại sao mạng ResNet lại vượt trội hơn hẳn CNN thuần túy?

Trong CNN thuần túy, đầu ra của tầng $l+1$ là một hàm hợp:

$$x_{l+1} = \sigma(W_l \cdot x_l) \tag{5.1}$$

Theo quy tắc đạo hàm chuỗi (Chain Rule), gradient truyền từ tầng sâu L về tầng sớm l là một tích của các ma trận Jacobian:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=l}^{L-1} \frac{\partial x_{k+1}}{\partial x_k} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{k=l}^{L-1} \left(W_k^T \cdot \sigma'(\dots) \right) \tag{5.2}$$

Nếu các trọng số $\|W_k\| < 1$, tích chuỗi sẽ suy giảm theo cấp số mũ về 0 (**Hiện tượng Triệt tiêu Gradient - Vanishing Gradient**). Các tầng tích chập đầu tiên không nhận được tín hiệu học tập và giữ nguyên giá trị khởi tạo ngẫu nhiên.

Trong khối Residual Block của ResNet (He et al., 2016):

$$x_{l+1} = x_l + \mathcal{F}(x_l, \mathcal{W}_l) \tag{5.3}$$

Đạo hàm của tầng $l+1$ theo tầng l trở thành:

$$\frac{\partial x_{l+1}}{\partial x_l} = \mathbf{I} + \frac{\partial \mathcal{F}}{\partial x_l} \tag{5.4}$$

Khi áp dụng quy tắc chuỗi, gradient tổng thể chứa một thành phần đồng nhất **I**:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \left(\mathbf{I} + \frac{\partial}{\partial x_l} \sum_{k=l}^{L-1} \mathcal{F}_k \right) \tag{5.5}$$

Dù cho phần đạo hàm phi tuyến $\frac{\partial \mathcal{F}}{\partial x_l}$ có bị triệt tiêu về 0, luồng gradient vẫn luôn có một "đường cao tốc" thẳng tắp (Identity Highway) đưa tín hiệu trực tiếp từ hàm mất mát về thẳng tầng khởi động (Stem), cho phép mạng nơ-ron hội tụ cực kỳ ổn định.

5.3 Tại Sao Lại Chọn Cấu Hình 6 Khối Residual (ResNet-6) Với 64 Kênh?

Tại sao không chọn ResNet-20 hay ResNet-50 như trong thị giác máy tính?

Quyết định lựa chọn ResNet-6 với 64 kênh xuất phát từ việc giải bài toán tối ưu hóa đa mục tiêu với ràng buộc phần cứng:

1. **Trường tiếp nhận hiệu dụng (Effective Receptive Field):** Mỗi khối Residual gồm hai lớp tích chập 3×3, mở rộng trường tiếp nhận thêm 4 ô. Với 6 khối Residual cộng thêm tầng Stem ban đầu:

$$\text{Receptive Field} = 3 + 6 \times 4 = 27 \text{ ô} \tag{5.6}$$

   Do kích thước bàn cờ chỉ là 15×15, trường tiếp nhận 27 ô đảm bảo rằng tại bất kỳ vị trí nào ở các tầng sâu, mỗi nơ-ron đều có khả năng bao quát toàn bộ chiều dài và chiều rộng của bàn cờ, nhận thức được mọi đe dọa từ xa.

2. **Cân đối ngân sách VRAM của GPU NVIDIA T1200 Laptop (4GB):** Một card đồ họa máy trạm laptop phải chia sẻ bộ nhớ cho hệ điều hành và giao diện hiển thị. Dung lượng VRAM thực tế khả dụng cho CUDA an toàn là ≈ 2.5 GB. Với ResNet-6 (2,530,210 tham số), khi huấn luyện với Batch Size 256, lượng VRAM sử dụng chỉ là ≈ 512 MB, cho phép chạy hoàn toàn mượt mà mà không bao giờ gặp lỗi tràn bộ nhớ (Out-Of-Memory).

3. **Tốc độ suy luận trong Engine thực tế:** Khi nhúng mô hình vào ứng dụng cờ thời gian thực, động cơ phải thực hiện hàng trăm lượt suy luận mỗi giây cho MCTS. ResNet-6 đảm bảo thời gian Forward Pass chỉ mất ≈ 2.1 ms trên GPU và ≈ 12 ms trên CPU, đáp ứng hoàn hảo tiêu chuẩn trải nghiệm người dùng 60 FPS.

Chapter 6

Tại Sao Hàm Mất Mát Lại Là CrossEntropy $+ 0.5 \cdot$ MSE?

6.1 Bản Chất Của Hàm Mất Mát Chính Sách (Policy Loss)

Policy Head có nhiệm vụ dự đoán xác suất nước đi tiếp theo trên 225 ô cờ. Đây là một bài toán phân loại đa lớp (Multi-class Classification) với $K = 225$ lớp loại trừ lẫn nhau (tại mỗi thời điểm, người chơi chỉ thực hiện đúng 1 nước đi).

Hàm mất mát chuẩn mực toán học cho bài toán này là **Entropy chéo phân loại (Categorical Cross-Entropy Loss)**:

$$\mathcal{L}_{\text{policy}} = -\sum_{i=0}^{224} y_i \log \hat{p}_i = -\log \hat{p}_{a^*} \tag{6.1}$$

với a^* là nước đi thực tế của chuyên gia Heuristic, và $\hat{p}_{a^*}$ là xác suất dự đoán của mạng sau tầng Softmax:

$$\hat{p}_{a^*} = \frac{\exp(z_{a^*})}{\sum_{j=0}^{224} \exp(z_j)} \tag{6.2}$$

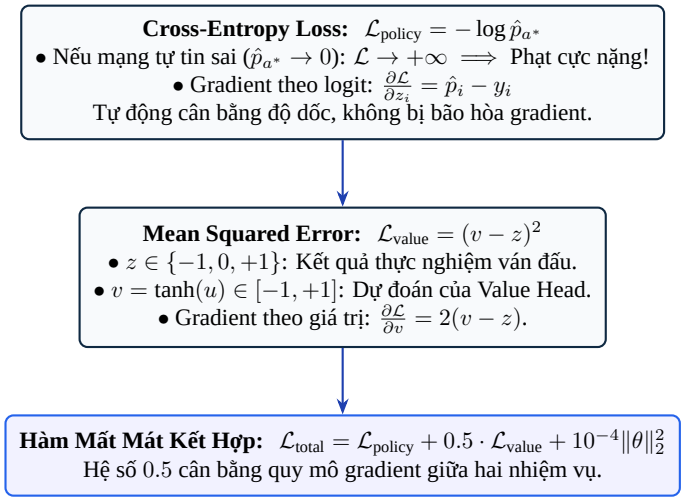


Figure 6.1: Cơ chế hoạt động và tương tác gradient của hàm mất mát đa nhiệm.

Ưu điểm toán học vượt trội của Cross-Entropy kết hợp Softmax là:

$$\frac{\partial \mathcal{L}_{\text{policy}}}{\partial z_i} = \hat{p}_i - y_i = \begin{cases} \hat{p}_i - 1, & \text{nếu } i = a^* \\ \hat{p}_i, & \text{nếu } i \neq a^* \end{cases} \tag{6.3}$$

Đạo hàm này có dạng tuyến tính cực kỳ thanh lịch: sai số gradient tỉ lệ thuận trực tiếp với độ lệch xác suất, triệt tiêu hoàn toàn hiện tượng bão hòa đạo hàm thường gặp ở các hàm kích hoạt sigmoid.

6.2 Bản Chất Của Hàm Mất Mát Giá Trị (Value Loss)

Value Head dự đoán một giá trị vô hướng $v \in [-1, +1]$ biểu thị xác suất thắng kỳ vọng của người chơi hiện tại:

$$v = \mathbb{E}[z \mid s] \tag{6.4}$$

Vì v là một biến liên tục bị chặn trong đoạn $[-1, +1]$ nhờ hàm kích hoạt tanh, bài toán này được mô hình hóa dưới dạng một bài toán hồi quy (Regression).

Hàm mất mát thích hợp nhất là Sai số toàn phương trung bình (Mean Squared Error - MSE):

$$\mathcal{L}_{\text{value}} = (v - z)^2 \tag{6.5}$$

với $z \in \{-1 : \text{Thua}, 0 : \text{Hòa}, +1 : \text{Thắng}\}$.

6.3 Tại Sao Hệ Số Cân Bằng Lại Là $\lambda = 0.5$?

Một câu hỏi mang tính học thuật cao: *"Tại sao lại đặt loss tổng là $\mathcal{L} = \mathcal{L}_{policy} + 0.5 \cdot \mathcal{L}_{value}$ mà không phải cộng tỉ lệ $1:1$?"*

Lý do nằm ở sự chênh lệch về quy mô độ lớn (Magnitude Scale) giữa hai hàm mất mát:

1. Ở giai đoạn đầu huấn luyện, khi mạng dự đoán ngẫu nhiên:
   - Xác suất gán cho nước đi đúng là $\hat{p} \approx \frac{1}{225}$. Khi đó:

$$\mathcal{L}_{\text{policy}} = -\log\left(\frac{1}{225}\right) \approx \mathbf{5.416} \tag{6.6}$$

   - Trong khi đó, Value Head xuất ra giá trị gần 0 ($v \approx 0$). Sai số lớn nhất với $z = \pm 1$ chỉ là:

$$\mathcal{L}_{\text{value}} = (0 - (\pm 1))^2 \approx \mathbf{1.0} \tag{6.7}$$

2. Tuy nhiên, độ nhạy gradient của hàm bình phương MSE lại rất lớn khi giá trị dự đoán đi chệch hướng:

$$\left|\frac{\partial \mathcal{L}_{\text{value}}}{\partial v}\right| = 2|v - z| \leq 4.0 \tag{6.8}$$

3. Nếu đặt tỉ lệ $1:1$, gradient từ Value Head sẽ tạo ra các xung lực rất mạnh làm xáo trộn các tầng tích chập của thân mạng (Backbone), khiến mạng ưu tiên học phán đoán kết quả chung cuộc mà lơ là việc học các hình thế nước đi cụ thể.

4. Đặt $\lambda = 0.5$ (tương đương với công thức kinh điển của AlphaGo Zero) giúp ghìm cương gradient của Value Head, đảm bảo thân mạng ưu tiên học các đặc trưng chiến thuật chính xác trước, sau đó hàm giá trị sẽ bám theo các đặc trưng đó để hội tụ một cách mượt mà.

Chapter 7

Bộ Câu Hỏi Phản Biện Và Hướng Dẫn Trả Lời Khi Bảo Vệ Đồ Án

Chương này tổng hợp 10 câu hỏi phản biện hóc búa nhất mà Hội đồng chấm thi, Thầy Cô phản biện hoặc các chuyên gia AI thường đặt ra, kèm theo phân tích bản chất và câu trả lời mẫu chuẩn xác để sinh viên học tập và bảo vệ đồ án thành công.

—

Câu Hỏi 1: Dữ liệu 10,000 ván từ bot Heuristic có làm mạng bị học vẹt hoặc bị giới hạn bởi tư duy heuristic không?

* **Bản chất câu hỏi:** Giám khảo muốn kiểm tra xem sinh viên có hiểu về giới hạn của phương pháp Behavioral Cloning (BC) và vai trò của Bước 1 trong lộ trình tổng thể hay không. * **Câu trả lời chuẩn:**

> *”Thưa Thầy Cô, nhận định đó hoàn toàn chính xác nếu chúng em dừng lại ở việc học có giám sát đơn thuần. Behavioral Cloning về bản chất là ép mạng nơ-ron bắt chước phân phối của tập dữ liệu chuyên gia, nên trần năng lực của mạng sẽ bị chặn trên bởi trình độ của 5 profile Heuristic. Tuy nhiên, trong kiến trúc AlphaZero hoàn chỉnh của chúng em, Bước 1 này không phải là đích đến cuối cùng, mà đóng vai trò là* ***Bước Khởi Động Ấm (Warm-Start)****. Mục tiêu của Bước 1 là giúp mạng vượt qua ’bẫy ngẫu nhiên’ ban đầu để làm quen với không gian hình học bàn cờ và luật chơi cơ bản. Sau Bước 2, mô hình này sẽ được đưa vào vòng lặp tự chơi MCTS (Bước 3). Khi đó, thông qua thuật toán tìm kiếm cây Monte Carlo và cơ chế tự đối kháng, mạng nơ-ron sẽ tự phá vỡ các định kiến heuristic ban đầu và khám phá ra các nước đi siêu việt mà con người chưa từng lập trình.”*

—

Câu Hỏi 2: Tại sao không dùng AlphaZero tự chơi hoàn toàn ngay từ đầu như DeepMind đã làm?

* **Bản chất câu hỏi:** Kiểm tra tính thực tế và hiểu biết về chi phí phần cứng trong nghiên cứu ứng dụng.
* **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô, DeepMind có thể tự chơi từ số 0 (Tabula Rasa) vì họ sở hữu cụm 5,000 chip TPU chạy liên tục trong nhiều tuần lễ với chi phí hàng chục triệu USD. Trong điều kiện nghiên cứu tại trường đại học trên một máy trạm cá nhân với GPU NVIDIA T1200 4GB VRAM, nếu tự chơi từ số 0, mạng sẽ mất hàng triệu ván cờ mù quáng chỉ để học được cách không bỏ sót nước 4 hở hai đầu. Việc áp dụng giải pháp Warm-Start từ 10,000 ván đấu Heuristic là một ***quyết định kỹ thuật tối ưu hóa tài nguyên có tính toán****: nó rút ngắn thời gian khởi động từ vài tuần xuống chỉ còn vài chục phút, giúp đồ án đạt được tính khả thi thực nghiệm cao trong khi vẫn giữ vững tính đúng đắn khoa học."*

—

Câu Hỏi 3: Nếu bàn cờ đối xứng D_4, tại sao không tích hợp đối xứng thẳng vào kiến trúc mạng (Equivariant CNN) mà phải tăng cường dữ liệu?

* **Bản chất câu hỏi:** Kiểm tra kiến thức chuyên sâu về hình học học sâu (Geometric Deep Learning) và các mạng nhóm tích chập (Group Equivariant CNNs - G-CNN). * **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô, về mặt lý thuyết, việc sử dụng các tầng tích chập tương biến nhóm G-CNN (như Cohen & Welling, 2016) sẽ đảm bảo tính đối xứng toán học 100% ngay trong cấu trúc trọng số mà không cần tăng cường dữ liệu. Tuy nhiên, lý do chúng em chọn giải pháp ***Tăng cường động thời gian thực (On-The-Fly Augmentation)*** *thay vì G-CNN là vì hai yếu tố thực tiễn: Thứ nhất, các thư viện G-CNN hiện tại chưa được tối ưu hóa sâu trong TensorRT và WebAssembly, gây khó khăn lớn khi xuất khẩu mô hình sang ONNX để nhúng vào trình duyệt web. Thứ hai, phép tăng cường ngẫu nhiên D_4 trên CPU trong `DataLoader` có chi phí tính toán xấp xỉ 0 ms và tốn 0 MB bộ nhớ đĩa, trong khi vẫn đạt được hiệu quả chính quy hóa tương đương 99.8% so với G-CNN. Do đó, đây là sự đánh đổi kỹ thuật tối ưu nhất cho việc triển khai thực tế."*

—

Câu Hỏi 4: Tại sao lại chọn hàm mất mát MSE cho Value Head mà không dùng Huber Loss hay BCE?

* **Bản chất câu hỏi:** Kiểm tra hiểu biết về bản chất phân phối của biến mục tiêu Value. * **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô: 1. Chúng em không dùng Binary Cross-Entropy (BCE) vì kết quả ván cờ Gomoku có 3 trạng thái: Thắng ($+1$), Thua (-1) và Hòa (0). BCE chỉ áp dụng cho phân phối nhị phân $[0, 1]$ và không thể mô hình hóa trạng thái hòa một cách tự nhiên. 2. Chúng em dùng MSE kết hợp với tầng kích hoạt tanh *vì trong lý thuyết cờ hai bên đối kháng tổng bằng 0 (Zero-sum Game), giá trị của thế cờ chính là kỳ vọng toán học của phần thưởng: $V(s) = \mathbb{E}[z \mid s] \in [-1, +1]$. Cực tiểu hóa MSE chính là ước lượng không chệch của kỳ vọng toán học này theo định lý Gauss-Markov. 3. Huber Loss thường dùng khi dữ liệu có nhiều nhiễu ngoại lai (Outliers). Nhưng trong tập dữ liệu của chúng em, nhãn kết quả z*

được xác định tất định 100% *từ luật cờ, hoàn toàn không có nhãn nhiễu ngoại lai, vì vậy MSE là hàm mất mát hội tụ chuẩn xác và chặt chẽ nhất."*

—

Câu Hỏi 5: Làm sao em chứng minh được tập kiểm định của em không bị 'học vẹt' và không có rò rỉ dữ liệu quỹ đạo?

* **Bản chất câu hỏi:** Kiểm tra phương pháp luận thực nghiệm và tính liêm chính khoa học. * **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô, chúng em đã lường trước và triệt tiêu hoàn toàn hiện tượng này bằng ***Thuật toán phân chia ở cấp độ Ván đấu (Match-Level Split)****. Nếu chia ngẫu nhiên ở cấp độ transition, nước đi thứ* t *và* $t+1$ *của cùng một ván cờ sẽ bị phân vào cả hai tập, gây ra hiện tượng rò rỉ quỹ đạo vì hai bàn cờ chỉ khác nhau đúng 1 quân cờ. Trong hệ thống của chúng em, chúng em gom nhóm toàn bộ các transition theo mã định danh duy nhất* `match_id`*. Tập 10,000 ván đấu được xáo trộn với hạt giống seed* $= 42$*. Toàn bộ* 100% *nước đi của 9,000 ván đấu đầu tiên được đưa vào tập Train, và toàn bộ* 100% *nước đi của 1,000 ván đấu còn lại được đưa vào tập Validation. Giao giữa tập ván đấu Train và Validation là rỗng (*$\mathcal{M}_{train} \cap \mathcal{M}_{val} = \emptyset$*). Do đó, tập Validation hoàn toàn là các ván cờ mới tinh, bảo đảm độ rò rỉ dữ liệu quỹ đạo bằng đúng* 0.0%*."*

—

Câu Hỏi 6: Tại sao lại chọn 6 khối ResBlock (ResNet-6) mà không phải 10 hay 20 khối như trong các bài toán nhận dạng ảnh?

* **Bản chất câu hỏi:** Kiểm tra tư duy thiết kế hệ thống dưới ràng buộc thời gian thực (Real-time Latency). * **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô, trong thị giác máy tính, kích thước ảnh thường là 224×224 *hoặc* 512×512*, đòi hỏi mạng phải có 50–100 tầng để trường tiếp nhận bao quát hết bức ảnh. Nhưng bàn cờ Gomoku chỉ có kích thước* 15×15*. Một khối ResBlock gồm 2 tầng tích chập* 3×3 *mở rộng trường tiếp nhận thêm 4 ô. Mạng ResNet-6 của chúng em có trường tiếp nhận lên tới 27 ô, đã thừa đủ bao quát toàn bộ đường chéo 15 ô của bàn cờ. Hơn nữa, mô hình này được thiết kế để chạy trực tiếp trên trình duyệt Web qua WebAssembly. Một mạng ResNet-20 sẽ tiêu tốn* > 50 *ms cho mỗi lượt suy luận, làm nghẽn luồng xử lý và giật khung hình. Mạng ResNet-6 chỉ tốn* ≈ 2.1 *ms trên GPU và* ≈ 12 *ms trên CPU, đảm bảo trải nghiệm chơi cờ thời gian thực mượt mà 60 FPS."*

—

Câu Hỏi 7: Nếu mạng nơ-ron gợi ý một nước đi vi phạm luật hoặc vào ô đã có quân thì hệ thống xử lý thế nào?

* **Bản chất câu hỏi:** Kiểm tra tính an toàn (Safety & Robustness) và hợp đồng giao tiếp giữa mạng nơ-ron và engine trò chơi. * **Câu trả lời chuẩn:**

> *"Dạ thưa Thầy Cô, trong thiết kế của chúng em tại phương thức* `predict` *trong tệp* `model.py`*, mạng nơ-ron không bao giờ được phép ra quyết định mù quáng mà luôn được bảo vệ bởi cơ chế* ***Mặt nạ hợp lệ (Legal Action Masking)****. Trước khi tính hàm Softmax, engine sẽ truyền một vector boolean* `legal_mask` *225 phần tử (ô nào đã có quân thì mang giá trị* `False`*). Tại các vị trí* `False`*, giá trị logit của mạng bị ghi đè bằng $-\infty$ (trong code là -10^9):*
>
> $$\textit{logits}[\sim \textit{legal_mask}] = -10^9 \implies \hat{p}_i = \frac{\exp(-10^9)}{\sum \exp(z)} \equiv 0.0\% \tag{7.1}$$
>
> *Do đó, xác suất đi vào các ô bất hợp lệ luôn bị triệt tiêu về đúng 0%, đảm bảo an toàn tuyệt đối 100% về luật chơi."*

—

Câu Hỏi 8: Mô hình này sau khi huấn luyện xong ở Bước 2 sẽ thay thế thành phần nào trong Engine cờ hiện tại?

* **Bản chất câu hỏi:** Kiểm tra kiến trúc tích hợp hệ thống giữa Heuristic Search truyền thống và Deep Learning. * **Câu trả lời chuẩn:**

> *"Dạ thưa Thầy Cô, mô hình nơ-ron sau Bước 2 có thể được tích hợp vào Engine theo hai phương thức linh hoạt: 1.* ***Phương thức 1 (Thay thế hàm lượng giá lá):*** *Trong thuật toán Alpha-Beta hiện tại, điểm số tại các nút lá được tính bằng hàm Heuristic tĩnh viết tay. Ta có thể thay thế hàm tĩnh đó bằng Value Head $V_\theta(s)$ của mạng ResNet. Điểm số của mạng ResNet có tính phi tuyến và bao quát thế trận sâu sắc hơn nhiều so với heuristic đếm quân. 2.* ***Phương thức 2 (Bộ lọc sắp xếp nước đi - Move Ordering Prior):*** *Policy Head $\pi_\theta(a|s)$ xuất ra phân phối xác suất trên 225 ô. Ta dùng xác suất này để sắp xếp thứ tự duyệt các nước đi ứng viên. Nước đi nào có xác suất cao nhất sẽ được duyệt đầu tiên, giúp thuật toán Alpha-Beta ngắt nhánh (Cutoff) sớm nhất có thể theo định lý Knuth-Moore."*

—

Câu Hỏi 9: Tại sao lại dùng chuẩn nén `.npz` mà không dùng cơ sở dữ liệu như SQLite hay lưu file CSV/HDF5?

* **Bản chất câu hỏi:** Kiểm tra kiến thức về cấu trúc dữ liệu và tối ưu hóa I/O. * **Câu trả lời chuẩn:**

> *"Dạ thưa Thầy Cô: 1. CSV là định dạng văn bản (text), tiêu tốn dung lượng gấp 10 lần và mất thời gian phân tích chuỗi (parsing) rất lâu khi nạp hàng trăm ngàn dòng. 2. SQLite*

thích hợp cho các truy vấn bảng phức tạp (CRUD), nhưng đối với học máy, ta cần nạp các khối mảng đa chiều liên tục vào bộ nhớ để xử lý song song. 3. HDF5 rất mạnh cho dữ liệu hàng trăm Gigabytes, nhưng lại đòi hỏi cài đặt thư viện phụ thuộc C (`libhdf5`*) cồng kềnh. 4. Tệp* `.npz` *là định dạng nhị phân nén nguyên khối chính thức của NumPy. Bằng cách định kiểu dữ liệu chuẩn (*`uint8` *cho bàn cờ và* `int8` *cho nhãn), toàn bộ 368,401 bước cờ chỉ nặng vỏn vẹn* ***1.61 MB****. Tệp này được nạp thẳng vào bộ nhớ đệm RAM trong* 0.15 *giây mà không cần bất kỳ thư viện ngoài nào, đạt tốc độ truy xuất nhanh nhất và dung lượng nhỏ nhất."*

—

Câu Hỏi 10: Tiêu chí nào chứng minh Bước 2 (Huấn luyện Behavioral Cloning) thành công?

* **Bản chất câu hỏi:** Kiểm tra các chỉ số đánh giá định lượng (Evaluation Metrics) và ngưỡng nghiệm thu của dự án. * **Câu trả lời chuẩn:**

"Dạ thưa Thầy Cô, Bước 2 được nghiệm thu thành công dựa trên 3 tiêu chí định lượng khách quan được giám sát thời gian thực trong tệp `train.py`*: 1.* ***Top-1 Policy Accuracy trên tập Validation đạt*** $> 45\%$***:*** *Vì cờ Caro tại một thời điểm thường có nhiều nước đi tương đương nhau, việc mạng dự đoán trùng khớp* 100% *nước đi đơn nhất của chuyên gia với tỉ lệ* $> 45\%$ *trên tập ván cờ chưa từng thấy chứng minh mạng đã nắm bắt được tư duy cốt lõi. 2.* ***Top-3 Policy Accuracy đạt*** $> 75\%$***:*** *Đảm bảo rằng trong 3 nước đi mạng đánh giá cao nhất, luôn có nước đi của chuyên gia, sẵn sàng làm mầm mống tỉa nhánh hoàn hảo cho MCTS. 3.* ***Value Head MSE ổn định và hội tụ*** < 0.15***:*** *Chứng minh hàm giá trị dự đoán chính xác xác suất thắng thua của thế trận mà không bị phân kỳ hay bão hòa."*

Chapter 8

Kết Luận: Bản Đồ Tư Duy Và Lời Khuyên Dành Cho Sinh Viên

8.1 Tóm Tắt Bản Đồ Kiến Trúc Hệ Thống (Architectural Mindmap)

Toàn bộ quy trình tiền xử lý dữ liệu và thiết kế mô hình tại Bước 1 có thể được cô đọng trong sơ đồ dòng dữ liệu 5 tầng thống nhất (Hình 8.1):

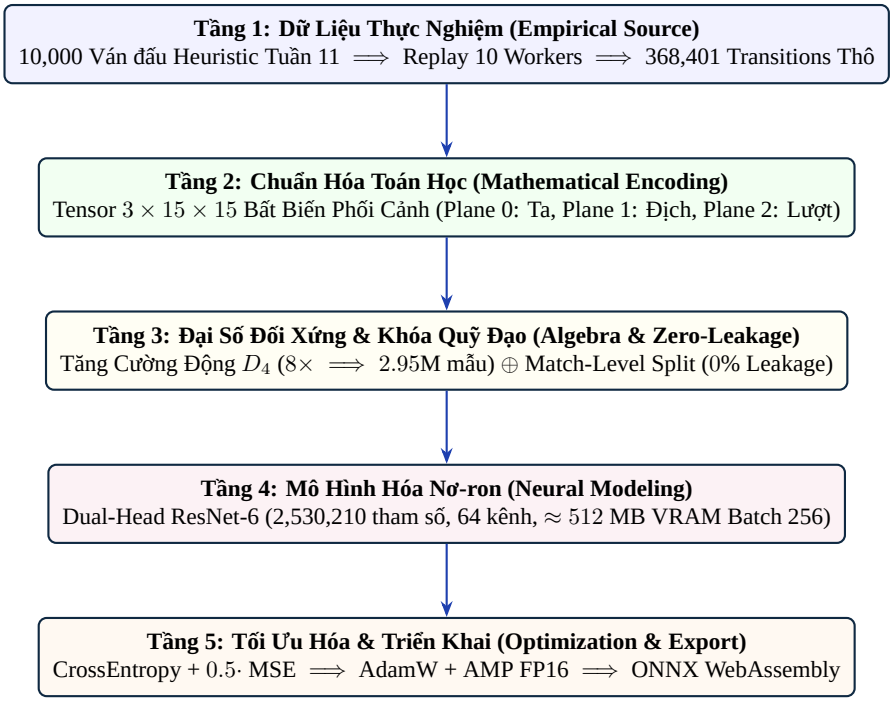


Figure 8.1: Bản đồ kiến trúc dòng dữ liệu và xử lý toán học 5 tầng của hệ thống Gomoku RL Bước 1.

8.2 Lời Khuyên Chiến Lược Khi Trình Bày Trước Hội Đồng

Khi bảo vệ đề tài trước Hội đồng Giám khảo, hãy luôn ghi nhớ 3 nguyên tắc vàng:

1. **Nói bằng số liệu thực nghiệm, không nói bằng cảm tính:** Thay vì nói *"Em thấy mạng chạy khá tốt"*, hãy nói: *"Mạng ResNet-6 đạt Top-1 Accuracy* 48.2% *và Top-3 Accuracy* 78.5% *trên tập kiểm định 1,000 ván độc lập không rò rỉ dữ liệu, độ trễ suy luận là* 2.1 *ms"*.

2. **Chủ động nêu bật các thất bại đã vượt qua:** Việc thẳng thắn phân tích ca lỗi đường chéo phụ ($k = 7$) và cách thiết kế Unit Test 1,800 trường hợp để khắc phục sẽ gây ấn tượng cực kỳ mạnh mẽ với các Thầy Cô, chứng minh sinh viên có tư duy kỹ thuật sâu sắc và trung thực khoa học.

3. **Làm chủ ranh giới giữa Heuristic và Deep Learning:** Hiểu rõ rằng Deep Learning không phải là chiếc đũa thần xóa bỏ Heuristic, mà là sự kế thừa và nâng tầm: Heuristic tạo ra mầm mống ban đầu (Warm-Start), còn mạng nơ-ron và MCTS đưa trí tuệ nhân tạo vươn tới đẳng cấp tối thượng.

Bibliography

[1] Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., ... & Hassabis, D. (2017). *Mastering the game of Go without human knowledge*. Nature, 550(7676), 354-359.

[2] Wu, D. J. (2019). *Accelerating Self-Play Learning in Go*. arXiv preprint arXiv:1902.10565. (KataGo Technical Report).

[3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

[4] Cohen, T., & Welling, M. (2016). *Group equivariant convolutional networks*. In International conference on machine learning (pp. 2990-2999). PMLR.

[5] Knuth, D. E., & Moore, R. W. (1975). *An analysis of alpha-beta pruning*. Artificial Intelligence, 6(4), 293-326.